

Technical Report: AI-Driven Self-Pruning Neural Networks

Case Study for Tredence Analytics

Konchada Shyam Sai
SRM Institute of Science and Technology

April 20, 2026

1 Introduction

Modern deep learning models often possess significant redundancy, making them inefficient for deployment on edge devices. This project implements a **Self-Pruning Neural Network** that dynamically learns to discard unnecessary connections during training. Unlike post-training pruning, this end-to-end approach integrates a learnable gating mechanism and specialized regularization to arrive at a sparse architecture automatically.

2 Problem Statement and Objectives

The objective was to build a system where each weight $w_{i,j}$ is associated with a learnable gate $g_{i,j} \in [0, 1]$. The goal is to maximize classification accuracy on the CIFAR-10 dataset while simultaneously minimizing the number of active gates, thereby achieving a high degree of model sparsity.

3 Technical Implementation Details

3.1 The PrunableLinear Module

The core of the architecture is the `PrunableLinear` class, which replaces the standard `nn.Linear` layer. It introduces several critical technical innovations:

- **Dual Parameterization:** Every weight matrix has a corresponding `gate_scores` tensor of identical shape. These scores are initialized to 0.0, ensuring the Sigmoid output starts at 0.5 (neutral).
- **Straight-Through Estimator (STE):** A major challenge in pruning is that thresholding (turning a soft gate into a 0 or 1) is non-differentiable. The implementation uses the STE trick:

$$\text{Gates} = (\text{HardMask} - \sigma(\text{Scores})).\text{detach}() + \sigma(\text{Scores}) \quad (1)$$

This ensures that the forward pass uses physical zeros (the `HardMask`), while the backward pass gradients flow through the differentiable Sigmoid function.

- **Initialization:** Weights use Kaiming Uniform initialization to maintain stable variance in deep ReLU networks.

3.2 Architecture for CIFAR-10

The model (CIFAR10PrunableNet) consists of a two-part pipeline:

1. **CNN Backbone:** Three convolutional layers (32, 64, and 128 filters) with Max-Pooling and ReLU activations. This reduces the $32 \times 32 \times 3$ input to a $128 \times 4 \times 4$ feature volume.
2. **Prunable Head:** The flattened 2048 features pass through two PrunableLinear layers with 512 and 10 neurons respectively. This is where the self-pruning logic is applied.

4 Mathematical Framework for Sparsity

4.1 Why L1 Regularization Encourages Sparsity

The total loss is defined as:

$$\mathcal{L}_{Total} = \mathcal{L}_{CrossEntropy} + \lambda \cdot \frac{1}{N} \sum_{i=1}^N |\sigma(\text{score}_i)| \quad (2)$$

The use of the $L1$ norm (sum of absolute values) is critical. Unlike $L2$ regularization, where the gradient decreases as values approach zero, the $L1$ norm has a constant gradient magnitude of 1. This exerts a persistent "pressure" that pushes gate scores toward $-\infty$ (resulting in a Sigmoid output of 0) unless the gate is essential for minimizing the classification loss.

4.2 Scale-Invariant Penalty

In the provided implementation, the sparsity loss is calculated as the **mean** of all gate values rather than the sum. This ensures that the λ hyperparameter remains effective regardless of changes in layer size or network depth, providing better engineering stability.

5 Experimental Results

The model was trained for 20 epochs using the Adam optimizer ($LR = 0.001$) and a batch size of 128. Training included data augmentation (Random Crops and Flips) to ensure the pruning was not simply memorizing noise.

5.1 Results Table

The following table summarizes the performance across different λ values:

Table 1: Impact of Lambda on Accuracy and Model Sparsity

Lambda (λ)	Test Accuracy (%)	Sparsity Level (%)	Status
0.0	82.49%	0.00%	Baseline
10.0	81.04%	52.85%	Moderate Pruning
50.0	80.68%	74.51%	High Pruning
100.0	80.91%	89.60%	Extreme Pruning

5.2 Analysis

At the highest λ of 100.0, the model successfully pruned **89.60% of its connections** in the prunable layers. Remarkably, the test accuracy only dropped from 82.49% to 80.91%, a marginal loss for a nearly 10x reduction in active parameters. This suggests high redundancy in the dense classification head.

6 Gate Value Distribution

In the best-performing pruned models, the gate distribution shows a classic bimodal pattern:

- **Primary Spike (0.0):** Representing the 89% of weights that were identified as non-contributory and effectively "switched off".
- **Secondary Cluster (1.0):** Representing the critical connections the network needs to maintain its 80%+ accuracy.

7 Conclusion

This implementation demonstrates the power of differentiable pruning. By utilizing learnable gates and the Straight-Through Estimator, the network effectively discovered a sub-network capable of matching baseline performance with only 10% of the original weights. This approach is highly suitable for production environments where model size and inference speed are paramount.